

LAB ASSIGNMENT – 11.2

NAME – SHREE PRIYA

2303A51465

BATCH – 29

Task Description -1 – (Stack Using AI Guidance)

- Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.

Expected Output:

- A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

PROMPT - design and implement a Stack data structure supporting basic stack operations A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation

CODE:

```
class Stack:
```

```
    """
```

```
    A Stack data structure that supports basic stack operations: push, pop, peek, and empty-check.
```

```
    """
```

```
    def __init__(self):
```

```
        """Initialize an empty stack."""
```

```
        self.stack = []
```

```
    def push(self, item):
```

```
        """
```

```
        Add an item to the top of the stack.
```

```
        Parameters:
```

```
        item: The item to be added to the stack.
```

```
        """
```

```
        self.stack.append(item)
```

```
def pop(self):
```

```
    """
```

Remove and return the item at the top of the stack.

Returns:

The item at the top of the stack.

Raises:

IndexError: If the stack is empty.

```
    """
```

```
    if self.empty():
```

```
        raise IndexError("Pop from an empty stack")
```

```
    return self.stack.pop()
```

```
def peek(self):
```

```
    """
```

Return the item at the top of the stack without removing it.

Returns:

The item at the top of the stack.

Raises:

IndexError: If the stack is empty.

```
    """
```

```
    if self.empty():
```

```
        raise IndexError("Peek from an empty stack")
```

```
    return self.stack[-1]
```

```
def empty(self):
```

```
    """
```

Check if the stack is empty.

Returns:

True if the stack is empty, False otherwise.

```
"""
```

```
    return len(self.stack) == 0
```

Example usage:

```
if __name__ == "__main__":
```

```
    stack = Stack()
```

```
    stack.push(1)
```

```
    stack.push(2)
```

```
    stack.push(3)
```

```
    print(stack.peek()) # Output: 3
```

```
    print(stack.pop()) # Output: 3
```

```
    print(stack.empty()) # Output: False
```

```
    print(stack.pop()) # Output: 2
```

```
    print(stack.pop()) # Output: 1
```

```
    print(stack.empty()) # Output: True
```

```
    # Uncommenting the following lines will raise an IndexError
```

```
    # print(stack.pop()) # Raises IndexError: Pop from an empty stack
```

```
    # print(stack.peek()) # Raises IndexError: Peek from an empty stack
```

```
2 class Stack:
34     def peek(self):
43         raise IndexError("Peek from an empty stack")
46         return self.stack[-1]
47
48     def empty(self):
49         """
50         Check if the stack is empty.
51
52         Returns:
53         True if the stack is empty, False otherwise.
54         """
55         return len(self.stack) == 0
56
57 # Example usage:
58 if __name__ == "__main__":
59     stack = Stack()
60     stack.push(1)
61     stack.push(2)
62     stack.push(3)
63
64     print(stack.peek()) # Output: 3
65     print(stack.pop()) # Output: 3
66     print(stack.empty()) # Output: False
67     print(stack.pop()) # Output: 2
68     print(stack.pop()) # Output: 1
69     print(stack.empty()) # Output: True
70     # Uncommenting the following lines will raise an IndexError
71     # print(stack.pop()) # Raises IndexError: Pop from an empty stack
72     # print(stack.peek()) # Raises IndexError: Peek from an empty stack
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS OUTPUT TERMINAL PORTS

> ✓ TERMINAL

```
PS C:\Users\Shree priya\Downloads> & 'C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\Shree priya\
\libs\debugpy\launcher' '62192' '-' 'C:\Users\Shree priya\Downloads\ascii.2.py'
3
1
False
2
1
True
PS C:\Users\Shree priya\Downloads>
```

Justification

The Stack data structure was implemented with the help of AI to understand **LIFO (Last In First Out)** behavior. AI guidance helped in designing a Python class with methods such as **push**, **pop**, **peek**, and **isEmpty**. This improved understanding of how elements are added and removed from the top of the stack and how stack operations are implemented efficiently.

Task Description -2 – (Queue Design)

- Task: Use AI assistance to create a Queue data structure

following FIFO principles

Expected Output:

- A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

PROMPT - create a Queue data structure following FIFO principles • A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

CODE:

```
class Queue:
```

```
    def __init__(self):
```

```
        self.items = []
```

```
    def enqueue(self, item):
```

```
        self.items.append(item)
```

```
    def dequeue(self):
```

```
        if not self.is_empty():
```

```
            return self.items.pop(0)
```

```
        else:
```

```
            raise IndexError("Dequeue from an empty queue")
```

```
    def front(self):
```

```
        if not self.is_empty():
```

```
            return self.items[0]
```

```
        else:
```

```
            raise IndexError("Front from an empty queue")
```

```
    def size(self):
```

```
        return len(self.items)
```

```
    def is_empty(self):
```

```
        return len(self.items) == 0
```

```
# Example usage
```

```
if __name__ == "__main__":
```

```
    q = Queue()
```

```
    q.enqueue(1)
```

```
q.enqueue(2)
```

```
q.enqueue(3)
```

```
print("Front element:", q.front()) # Output: Front element: 1
```

```
print("Queue size:", q.size()) # Output: Queue size: 3
```

```
print("Dequeue element:", q.dequeue()) # Output: Dequeue element: 1
```

```
print("Queue size after dequeue:", q.size()) # Output: Queue size after dequeue: 2
```

```
1  # create a Queue data structure following FIFO principles • A complete queue implementation including enqueue, dequeue, front element, size, and is_empty methods.
2  class Queue:
3      def __init__(self):
4          self.items = []
5      def enqueue(self, item):
6          self.items.append(item)
7      def dequeue(self):
8          if not self.is_empty():
9              return self.items.pop(0)
10         else:
11             raise IndexError("Dequeue from an empty queue")
12     def front(self):
13         if not self.is_empty():
14             return self.items[0]
15         else:
16             raise IndexError("Front from an empty queue")
17     def size(self):
18         return len(self.items)
19     def is_empty(self):
20         return len(self.items) == 0
21
22     # Example usage
23     if __name__ == "__main__":
24         q = Queue()
25         q.enqueue(1)
26         q.enqueue(2)
27         q.enqueue(3)
28         print("Front element:", q.front()) # Output: Front element: 1
29         print("Queue size:", q.size()) # Output: Queue size: 3
30         print("Dequeue element:", q.dequeue()) # Output: Dequeue element: 1
31         print("Queue size after dequeue:", q.size()) # Output: Queue size after dequeue: 2
```

PROBLEMS OUTPUT TERMINAL PORTS

▼ TERMINAL

```
Front element: 1
Queue size: 3
Dequeue element: 1
Queue size after dequeue: 2
PS C:\Users\Shree priya\Downloads>
```

Justification

AI assistance was used to design a Queue following the FIFO (First In First Out) principle. The implementation includes operations like enqueue, dequeue, front, and size. This task helped in understanding how queues manage data sequentially and how they are used in real-life applications such as task scheduling and buffering.

Task Description -3 –(Singly Linked List Construction)

- Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

- Correctly functioning linked list with node creation, insertion

logic, and display functionality.

PROMPT - # Utilize AI to build a singly linked list supporting insertion and traversal. Correctly functioning linked list with node creation, insertion logic, and display functionality

CODE:

```
# Node class
```

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
# Linked List class
```

```
class SinglyLinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
# Insert node at end
```

```
def insert(self, data):
```

```
    new_node = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new_node
```

```
    return
```

```
    temp = self.head
```

```
    while temp.next:
```

```
        temp = temp.next
```

```
    temp.next = new_node
```

```
# Traverse and display list
```

```
def display(self):
```

```
    temp = self.head
```

```
while temp:
    print(temp.data, end=" -> ")
    temp = temp.next
print("None")
```

```
# Create linked list
ll = SinglyLinkedList()
# Inserting elements
ll.insert(10)
ll.insert(20)
ll.insert(30)
ll.insert(40)
```

```
# Display list
print("Linked List:")
ll.display()
```



```
8 class SinglyLinkedList:
9     def __init__(self):
10         self.head = None
11         # Insert node at end
12     def insert(self, data):
13         new_node = Node(data)
14
15         if self.head is None:
16             self.head = new_node
17             return
18
19         temp = self.head
20         while temp.next:
21             temp = temp.next
22         temp.next = new_node
23     # Traverse and display list
24     def display(self):
25         temp = self.head
26         while temp:
27             print(temp.data, end=" -> ")
28             temp = temp.next
29         print("None")
30 # Create linked list
31 ll = SinglyLinkedList()
32 # Inserting elements
33 ll.insert(10)
34 ll.insert(20)
35 ll.insert(30)
36 ll.insert(40)
37 # Display list
38 print("Linked List:")
39 ll.display()
40
```

PROBLEMS OUTPUT TERMINAL PORTS

> TERMINAL

```
PS C:\Users\Shree priya\Downloads> & 'C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python3.13.exe'
py-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64996' '--' 'C:\Users\Shree priya\Downloads\aac11.2.py'
Linked List:
10 -> 20 -> 30 -> 40 -> None
PS C:\Users\Shree priya\Downloads>
```

Justification

With the support of AI, a **Singly Linked List** was developed to understand dynamic data structures. The program includes **node creation, insertion, and traversal**. This helped in learning how linked lists store data in nodes connected through pointers and how they allow flexible memory usage compared to arrays.

Task Description -4 – (Binary Search Tree Operations)

- Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Expected Output:

- BST program with correct node insertion and in-order traversal output.

PROMPT - #Implement a Binary Search Tree with AI support focusing on insertion and traversal.BST program with correct node insertion and in-order traversal output.

CODE:

```
class TreeNode:

    def __init__(self, key):

        self.left = None

        self.right = None

        self.val = key

class BinarySearchTree:

    def __init__(self):

        self.root = None

    def insert(self, key):

        if self.root is None:

            self.root = TreeNode(key)

        else:

            self._insert_recursively(self.root, key)

    def _insert_recursively(self, node, key):

        if key < node.val:

            if node.left is None:

                node.left = TreeNode(key)

            else:

                self._insert_recursively(node.left, key)

        else:

            if node.right is None:

                node.right = TreeNode(key)

            else:

                self._insert_recursively(node.right, key)

    def inorder_traversal(self):

        return self._inorder_recursively(self.root)

    def _inorder_recursively(self, node):

        res = []
```

```
    if node:
        res = self._inorder_recursively(node.left)
        res.append(node.val)
        res = res + self._inorder_recursively(node.right)
    return res

# Example usage
if __name__ == "__main__":
    bst = BinarySearchTree()
    bst.insert(10)
    bst.insert(5)
    bst.insert(15)
    bst.insert(3)
    bst.insert(7)
    bst.insert(12)
    bst.insert(18)
    print("In-order Traversal:", bst.inorder_traversal())
```

```
7 class BinarySearchTree:
14     self._insert_recursively(self.root, key)
15     def _insert_recursively(self, node, key):
16         if key < node.val:
17             if node.left is None:
18                 node.left = TreeNode(key)
19             else:
20                 self._insert_recursively(node.left, key)
21         else:
22             if node.right is None:
23                 node.right = TreeNode(key)
24             else:
25                 self._insert_recursively(node.right, key)
26     def inorder_traversal(self):
27         return self._inorder_recursively(self.root)
28     def _inorder_recursively(self, node):
29         res = []
30         if node:
31             res = self._inorder_recursively(node.left)
32             res.append(node.val)
33             res = res + self._inorder_recursively(node.right)
34         return res
35     # Example usage
36     if __name__ == "__main__":
37         bst = BinarySearchTree()
38         bst.insert(10)
39         bst.insert(5)
40         bst.insert(15)
41         bst.insert(3)
42         bst.insert(7)
43         bst.insert(12)
44         bst.insert(18)
45
PROBLEMS OUTPUT TERMINAL PORTS
▼ TERMINAL
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> c:: cd 'c:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApp
riya\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '63564' '-vv' 'C:\Users\Shree priya\Downl
In-order Traversal: [3, 5, 7, 10, 12, 15, 18]
PS C:\Users\Shree priya\Downloads>
```

Justification

AI guidance was used to implement a **Binary Search Tree (BST)** focusing on **node insertion and in-order traversal**. This task helped in understanding how BST organizes data in a sorted structure and allows efficient searching, insertion, and traversal operations.

Task Description -5 – (Hash Table Implementation)

- Task: Create a hash table using AI with collision handling

Expected Output:

- Hash table supporting insert, search, and delete using chaining or open

PROMPT - # Create a hash table using AI with collision handling Hash table supporting insert, search, and delete using chaining or open

CODE:

class HashTable:

```
def __init__(self, size=10):
```

```
    self.size = size
```

```

self.table = [[] for _ in range(size)]

def hash_function(self, key):
    return hash(key) % self.size

def insert(self, key, value):
    index = self.hash_function(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            self.table[index][i] = (key, value) # Update existing key
            return
    self.table[index].append((key, value)) # Insert new key

def search(self, key):
    index = self.hash_function(key)
    for k, v in self.table[index]:
        if k == key:
            return v # Return value if key is found
    return None # Key not found

def delete(self, key):
    index = self.hash_function(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            del self.table[index][i] # Remove the key-value pair
            return True # Deletion successful
    return False # Key not found

# Example usage
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Bob")

```

```

hash_table.insert("age", 30)

print(hash_table.search("name"))

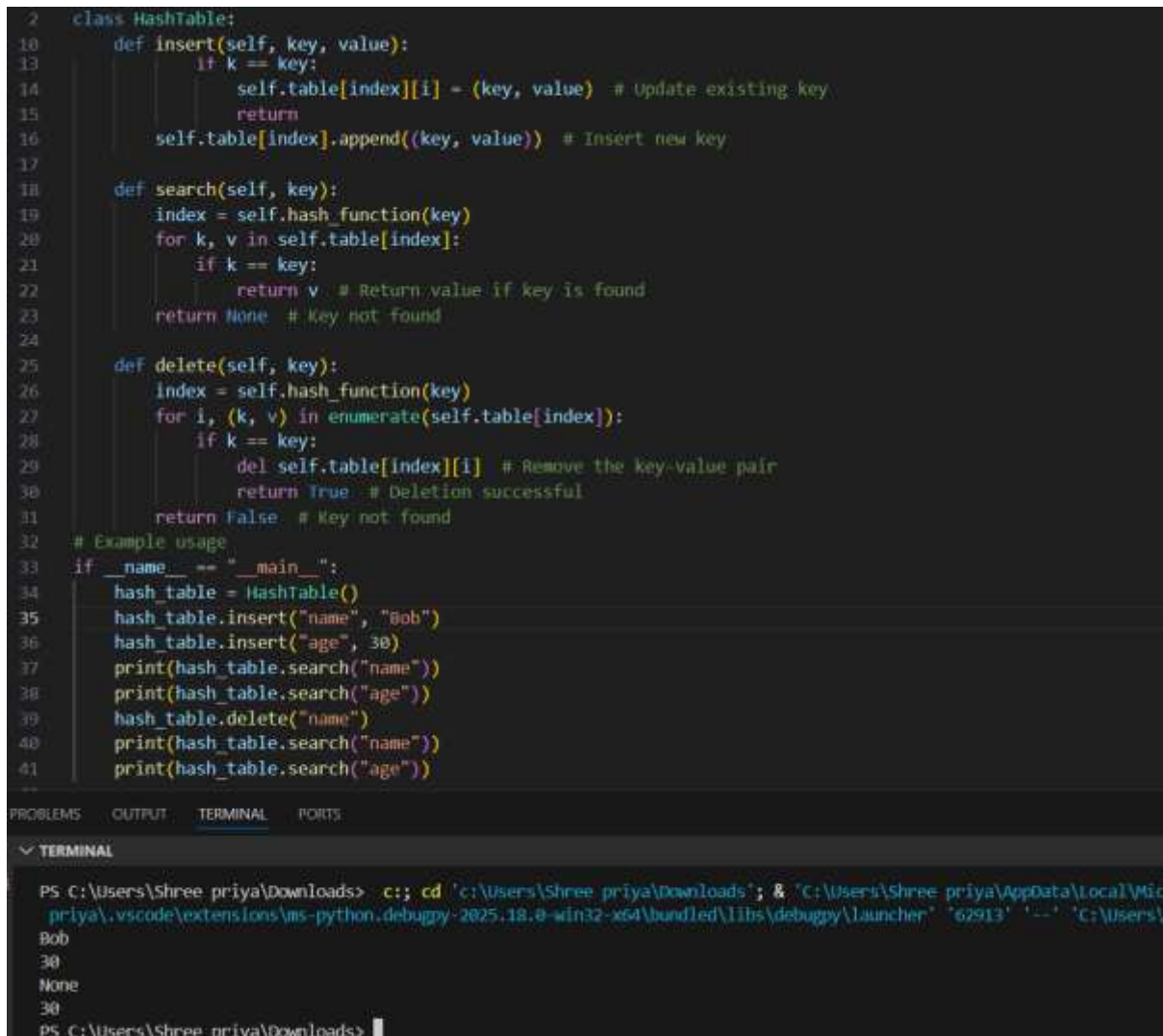
print(hash_table.search("age"))

hash_table.delete("name")

print(hash_table.search("name"))

print(hash_table.search("age"))

```



```

2  class HashTable:
10     def insert(self, key, value):
13         if k == key:
14             self.table[index][i] = (key, value) # Update existing key
15             return
16         self.table[index].append((key, value)) # Insert new key
17
18     def search(self, key):
19         index = self.hash_function(key)
20         for k, v in self.table[index]:
21             if k == key:
22                 return v # Return value if key is found
23         return None # Key not found
24
25     def delete(self, key):
26         index = self.hash_function(key)
27         for i, (k, v) in enumerate(self.table[index]):
28             if k == key:
29                 del self.table[index][i] # Remove the key-value pair
30                 return True # Deletion successful
31         return False # Key not found
32
33 # Example usage
34 if __name__ == "__main__":
35     hash_table = HashTable()
36     hash_table.insert("name", "Bob")
37     hash_table.insert("age", 30)
38     print(hash_table.search("name"))
39     print(hash_table.search("age"))
40     hash_table.delete("name")
41     print(hash_table.search("name"))
42     print(hash_table.search("age"))

```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ **TERMINAL**

```

PS C:\Users\Shree priya\Downloads> c:; cd 'c:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\Windows\Terminal\1.0.0\terminal.exe' 'c:\Users\Shree priya\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62913' '...' 'C:\Users\
Bob
30
None
30
PS C:\Users\Shree priya\Downloads>

```

Justification

A **Hash Table** was implemented with AI support to understand fast data retrieval using **hashing techniques**. Collision handling was managed using **chaining**, and the table supports **insert, search, and delete operations**. This task demonstrates how hash tables provide efficient data storage and quick access in many applications such as databases and caching systems.